

Deep Learning Optimized on Jean Zay

Dataset optimization

Storage spaces and data format

cnrs IDRIS

CC BY NC SA

Dataset optimization

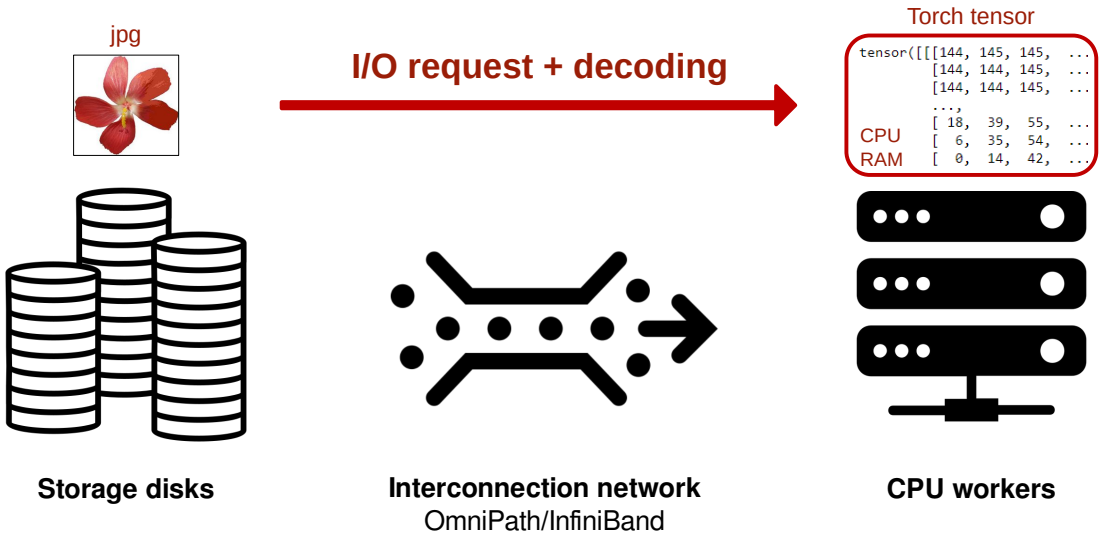
Main bottlenecks ◂

Data storage ◂

Data format ◂

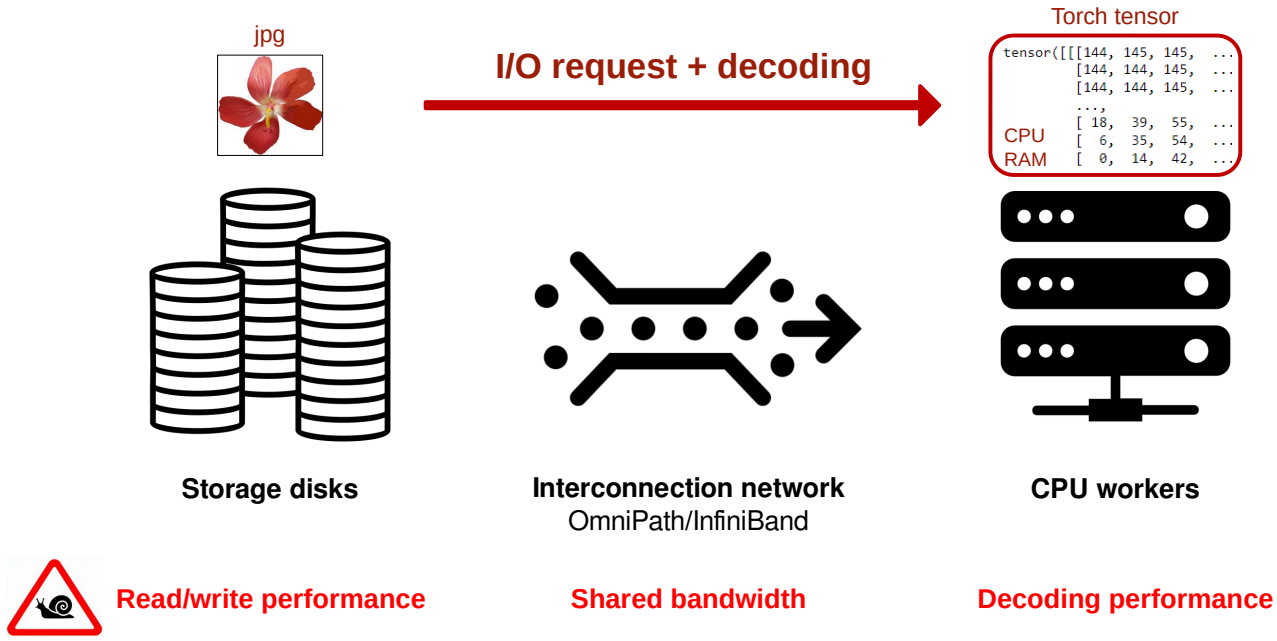
Dataset format ◂

Focus on I/O requests and data decoding



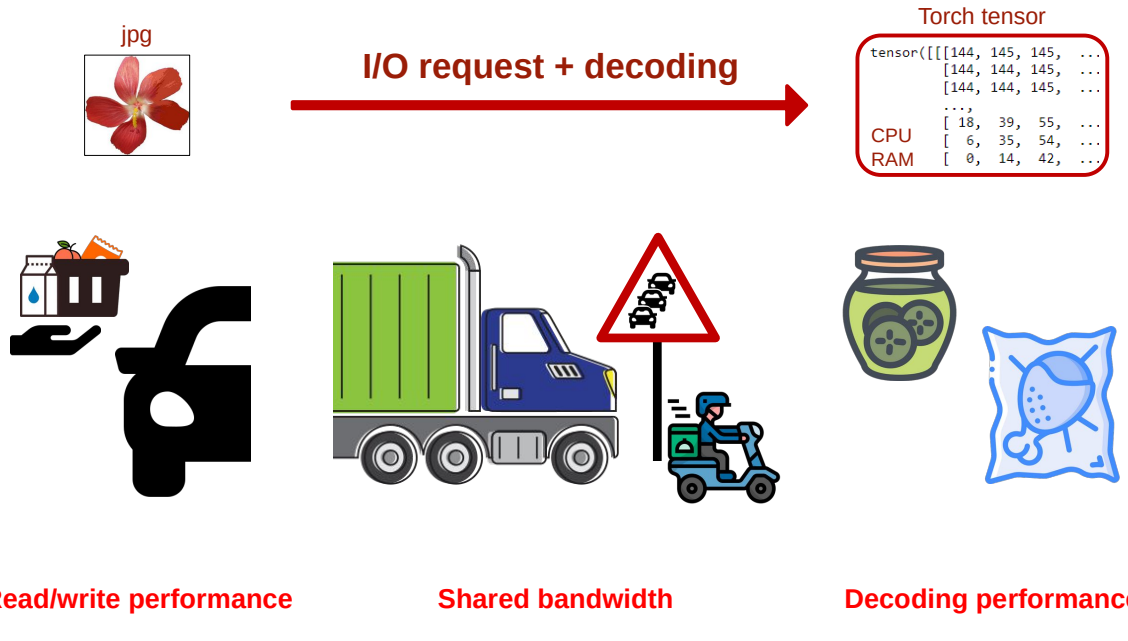
In this section, we will present different optimization possibilities in regard to the input data format and storage. This will impact the performance of I/O requests and data decoding.

Focus on I/O requests and data decoding



We are seeking to avoid three main obstacles: Slow I/O read speeds on disks, bandwidth sharing in the interconnection network, and the cost of input data decoding on the CPU.

Focus on I/O requests and data decoding





Read/write performance **Shared bandwidth** **Decoding performance**

To understand the various bottlenecks encountered, here is an analogy: reading data from a storage disk is like collecting your groceries at a drive-through.

Once at the drive-through, the staff will be more or less quick to serve you (analogy with the performance of different types of storage disks).

On the way back, there may be more or fewer people with you on the road, and even possible traffic jams (analogy with bandwidth sharing on the interconnect network).

If you collect fewer products at a time and travel by scooter, you will experience less traffic congestion but will have to make more trips back and forth.

If you collect many products at a time and travel by semi-trailer, you will experience traffic jams but won't need to return to the drive-through as often.

Once at home, you will eat more or less quickly, depending on the cooking time required for the type of food you have chosen (analogy with the more or less transformed data format).

Focus on I/O requests and data decoding

The efficiency of I/O operations and data decoding depends on both the **data format** and the **storage strategy**.

- **Where should I store my dataset?**
- **Which format for my data?**
- **Which format for my dataset?**

In this chapter, we will explore three key questions:

– Where should I store my dataset?
– Which format for my data?
– Which format for my dataset?

Dataset optimization

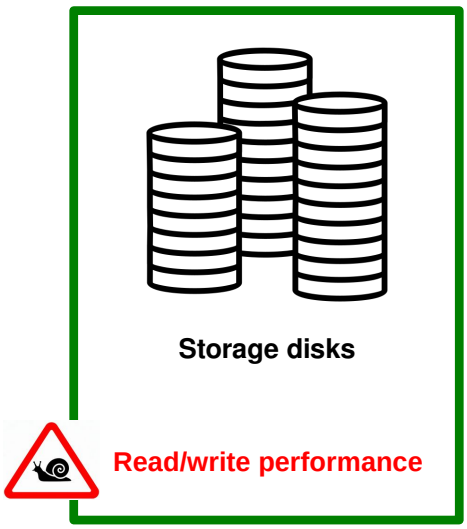
Main bottlenecks ◂

Data storage ◂

Data format ◂

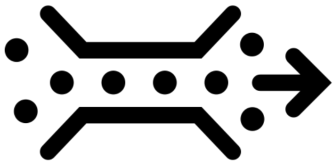
Dataset format ◂

Focus on I/O requests and data decoding



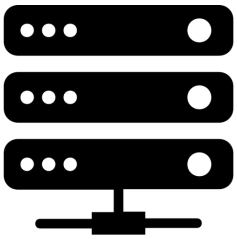
Storage disks

Read/write performance



Interconnection network
OmniPath/InfiniBand

Shared bandwidth



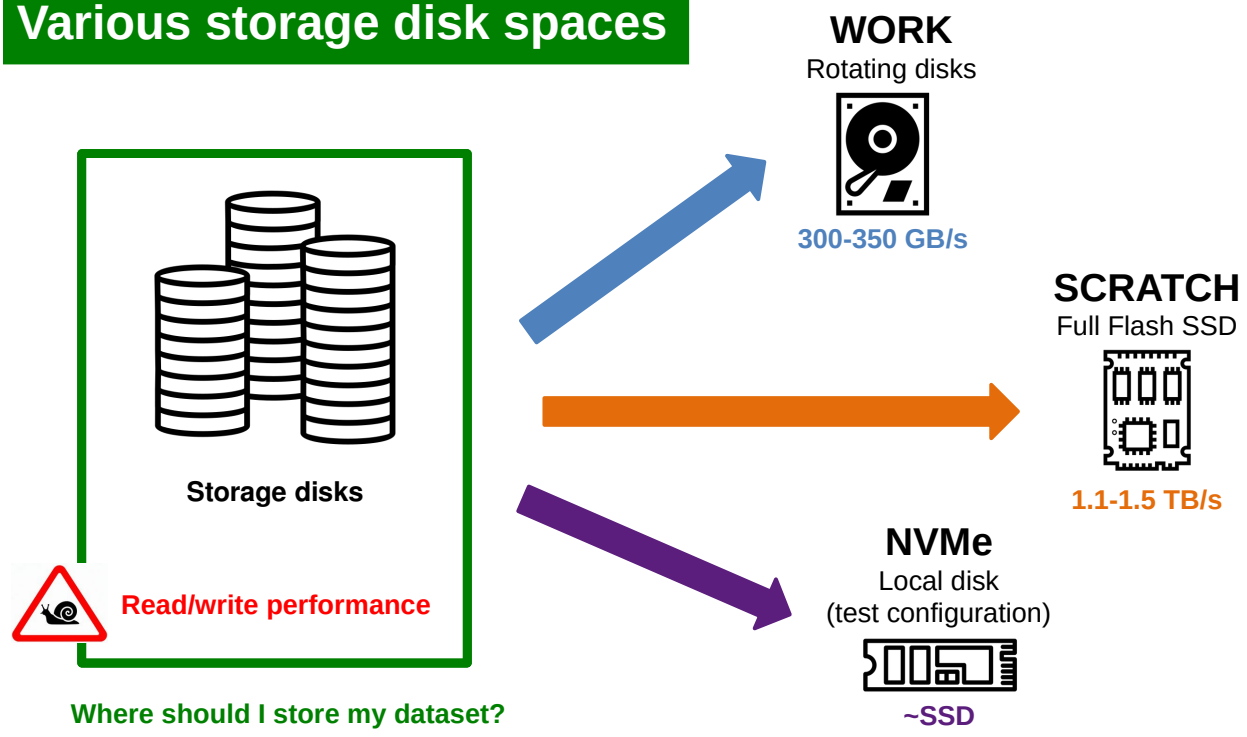
CPU workers

Decoding performance

Where should I store my dataset?

There are different storage technologies on Jean Zay. Where is it pertinent to store your input data?

Various storage disk spaces



The WORK space stores information on rotating disks with a read speed of 350 GB/s (300 GB/s in writing).

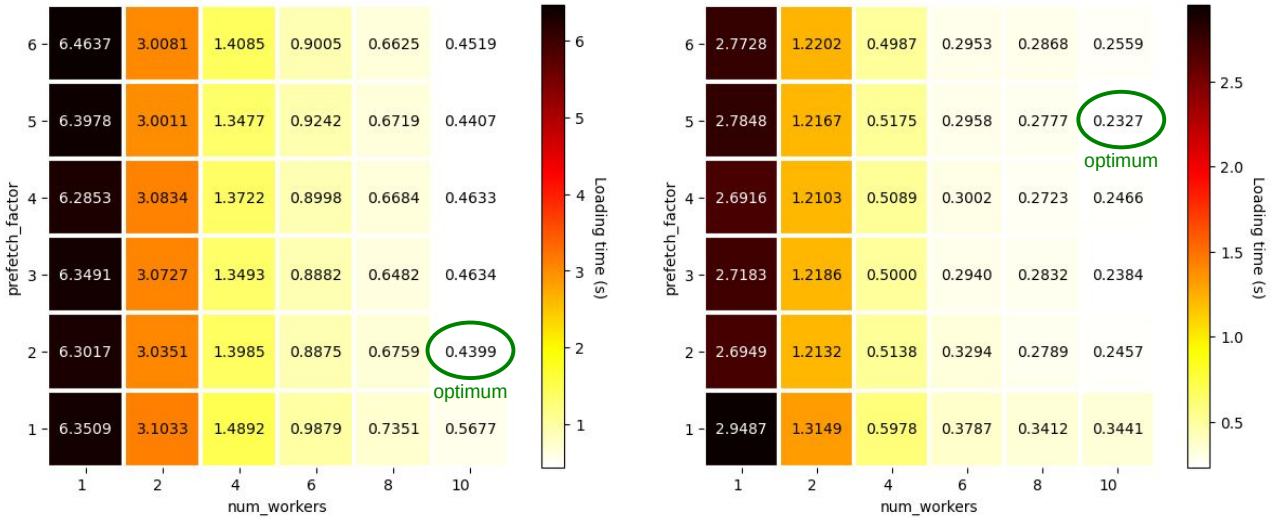
The SCRATCH space stores information on Full Flash SSD disks with a read speed of 1.5 TB/s (1.1 TB/s in writing), which is 4 times faster than with rotating disks.

It is possible to connect local SSD disks (NVMe) directly to a compute node, but this technology is not manageable for a supercomputer, and is only available internally for tests of technology watch. This avoids the bottleneck caused by traversing the shared interconnection network (OmniPath/InfiniBand). Information is shown for your personal knowledge.

Various storage disk spaces

dlojz.py - 50 iterations - test partition (A100 PCIe)

Historical benchmark - 2023 infrastructure. These absolute results should not be interpreted as current performance.



WORK
300-350 GB/s

SCRATCH
1.1-1.5 TB/s

NOTE: Test performed in 2023, when Jean Zay had a GPFS parallel file system and older-generation storage.

Here we ran the dlojz.py script during 50 iterations on an A100 PCIe GPU (test partition equipped with NVMe local disks).

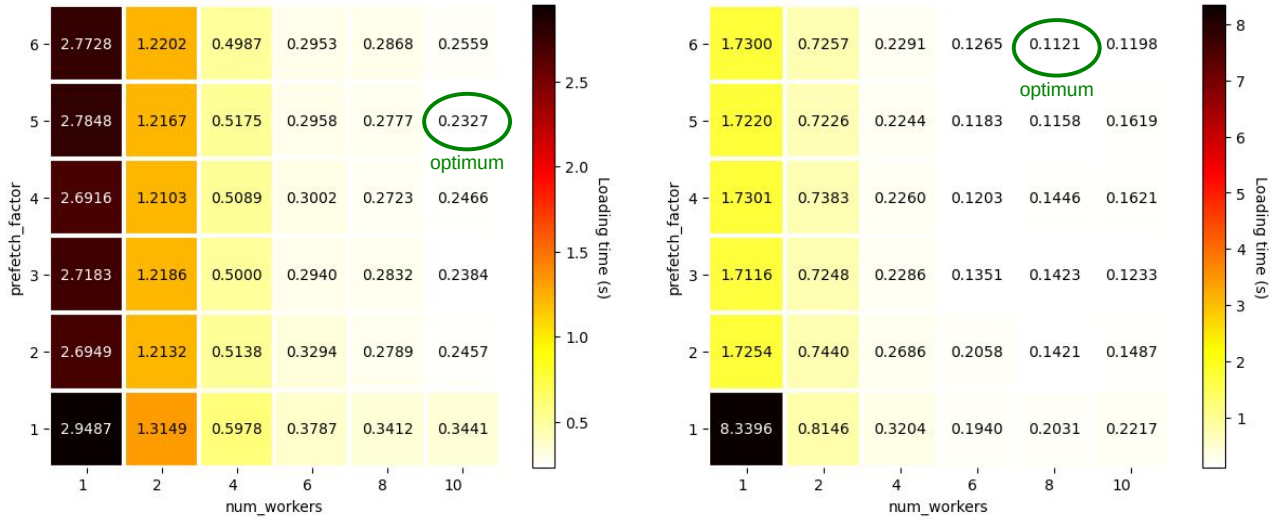
We report the loading time (DataLoader) in relation to the storage technology used, the number of workers and the prefetch_factor.

We observe that the time is halved when we store the database in the SCRATCH rather than in the WORK.

Various storage disk spaces

dlojz.py - 50 iterations - test partition (A100 PCIe)

Historical benchmark - 2023 infrastructure. These absolute results must not be interpreted as current performance.



SCRATCH

1.1-1.5 TB/s

NVMe

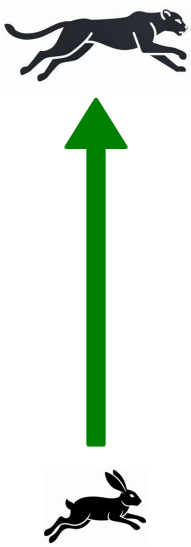
~SSD

We observe that the time is reduced by half when we store the database on NVMe rather than on the SCRATCH.

Reminder: using NVMe storage avoids the bottleneck caused by traversing the shared interconnection network (OmniPath/InfiniBand). However, this technology is not available on Jean Zay. Results are shown for your personal knowledge.

Various storage disk spaces

I/O speed



- **NVMe**
  - ✓ Best I/O performance
  - – You need to copy your dataset to the local disk first, which can take a very long time
  - – This solution is not suitable for supercomputer scale, so it is not available to users
- **SCRATCH**
  - ✓ Best possible I/O performance on Jean Zay
  - ✓ Very large quota (safety quota: ~500 TB / 150M inodes per project)
  - – 30 days file lifespan
- **WORK**
  - – Worst performance (but not bad!)
  - – Only 5 TB and 500k inodes per project
  - ✓ No automatic deletion procedure

The advantages and disadvantages of the different storage spaces.

Various storage disk spaces

- **Don’t forget the DSDIR for public datasets or models**!
  - SSD storage technology (similar to the SCRATCH but older generation)
  - Storage space accessible to all users
  - Downloads are handled by user support upon request



- **What about the STORE?**
  - Magnetic tape storage (with a cache on rotating disk)
  - 50 TB and 100k inodes per project
  - Intended to receive a small number of very large files (up to 10 TiB per file) → **archives**
  - Not accessible from the compute nodes (very high read/write latency)



Good practices

- Pool resources in the **DSDIR** if possible
- If you don’t need I/O performance (when building your workflow, debugging, or simply if I/O is not a bottleneck), store your dataset in the **WORK** if it is not too large
- If you need I/O performance or if the dataset is too large, store your dataset in the **SCRATCH**
- Keep a duplicate of your dataset as an archive in the **STORE**

Reminders about the DSDIR and STORE spaces, and summary of good practices.

Dataset optimization

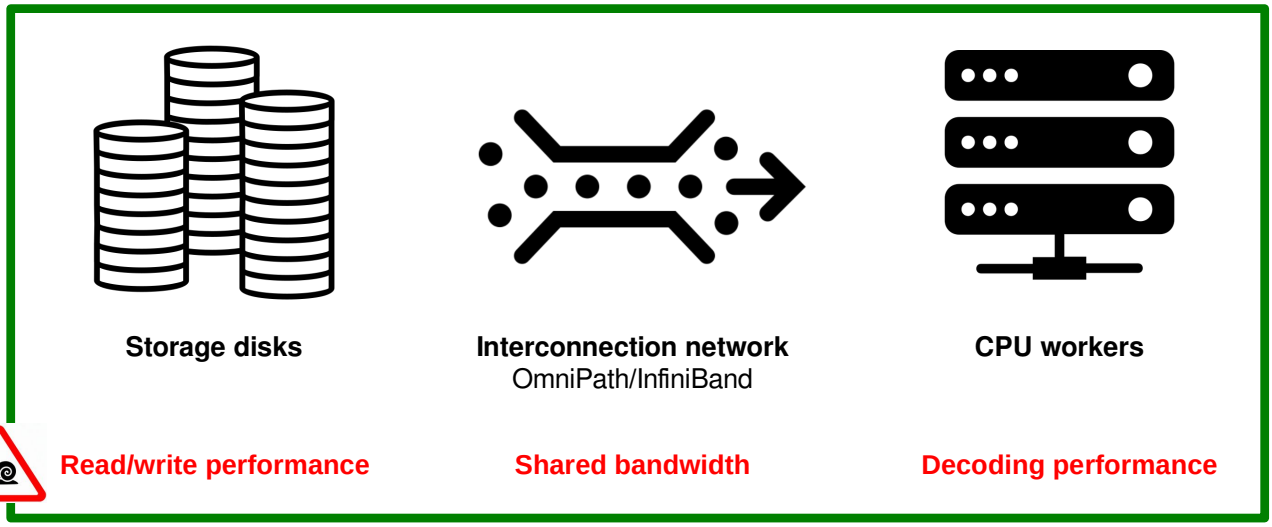
Main bottlenecks ◄

Data storage ◄

Data format ◄

Dataset format ◄

Focus on I/O requests and data decoding



Which format for my data?

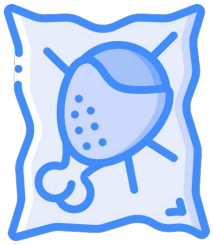
The input data format will have an impact on all the steps.

Data format



Binary format: Pickle, HDF5,...
Decoded more quickly, takes more space

- ✓ Decoder performance
- – Storage volume



Compressed format: jpeg, png,...
Decoded more slowly, takes less space

- – Decoder performance
- ✓ Storage volume

The binary formats enable a rapid decoding on CPU but use more disk space (~10x).

The compressed formats reduce the necessary storage volume but the decoding time on CPU is longer.

Dataset optimization

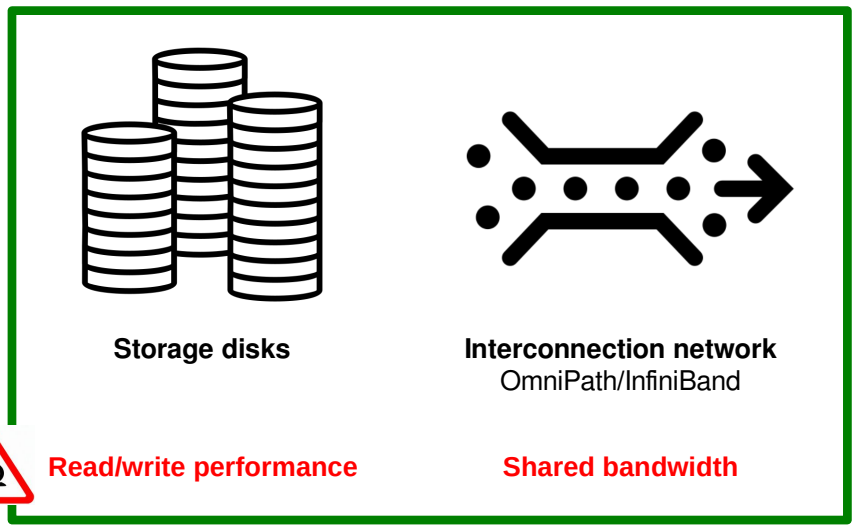
Main bottlenecks ◄

Data storage ◄

Data format ◄

Dataset format ◄

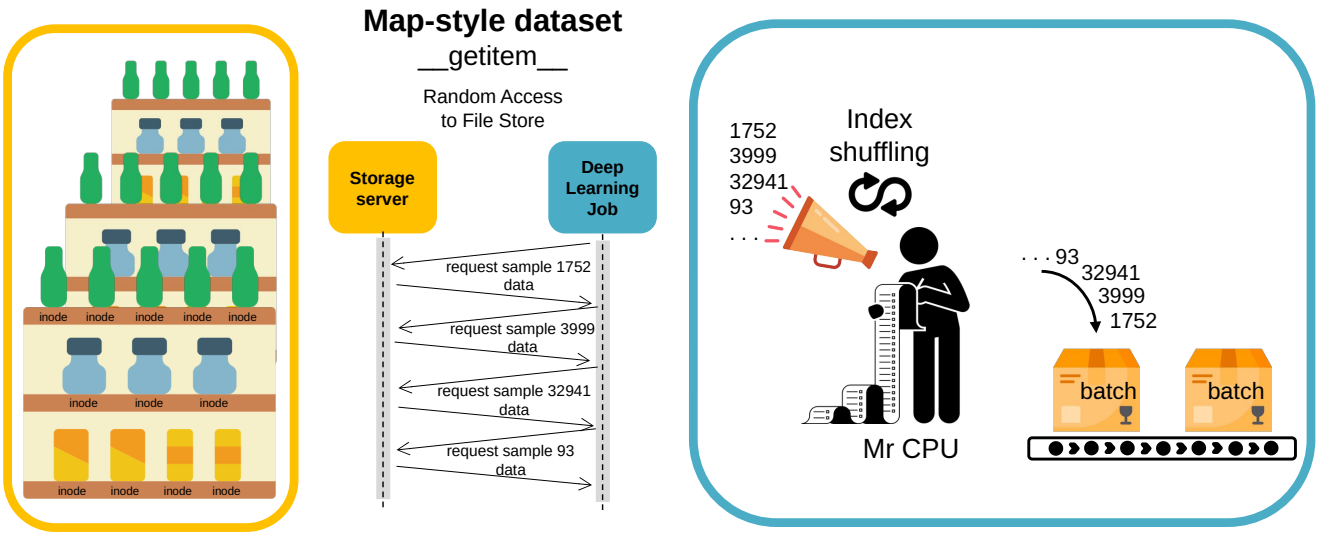
Focus on I/O requests and data decoding



Which format for my dataset?

Which format should I use for my database?

Intuitive way: 1 sample = 1 file



Pros: Easy to handle, random access possible

Cons: Lots of inodes, lots of I/Os

Here: 1 image = 1 file.

A map-style dataset corresponds to a dictionary (idx, image).

We access one data at a time from its index.

We generate one I/O request per each accessed data.

The shuffling is carried out on the indexes of the whole dataset before creating the batches.

Too many inodes are an issue

Error: Disk quota exceeded 😱

- The WORK quota per project is 5 TB / **500k inodes**
- The SCRATCH safety quota per project is ~500 TB / **150M inodes**

+ Parallel file system does not handle well:

- Intensive I/O workloads involving many small files
- A large number of files in a single directory





Reminder of the inode quotas on Jean Zay's disk spaces.

Parallel file systems are designed to optimize parallel access to the same large file. It adapts to new practices but is still suboptimal for I/O-intensive workloads with small files. Furthermore, it struggles to manage folders containing a very large number of files.

Too many inodes are an issue

- IDRIS recommendation on Jean Zay: **less than 100k inodes per directory**
- Repository limitations and recommendations on HuggingFace Hub

Characteristics	Recommended	Tips
Files per repo	<100k	merge data into fewer files
Entries per folder	<10k	use subdirectories in repo

https://huggingface.co/docs/hub/storage-limits#repository-limitations-and-recommendations

+ for large datasets, **WebDataset**-like format is recommended

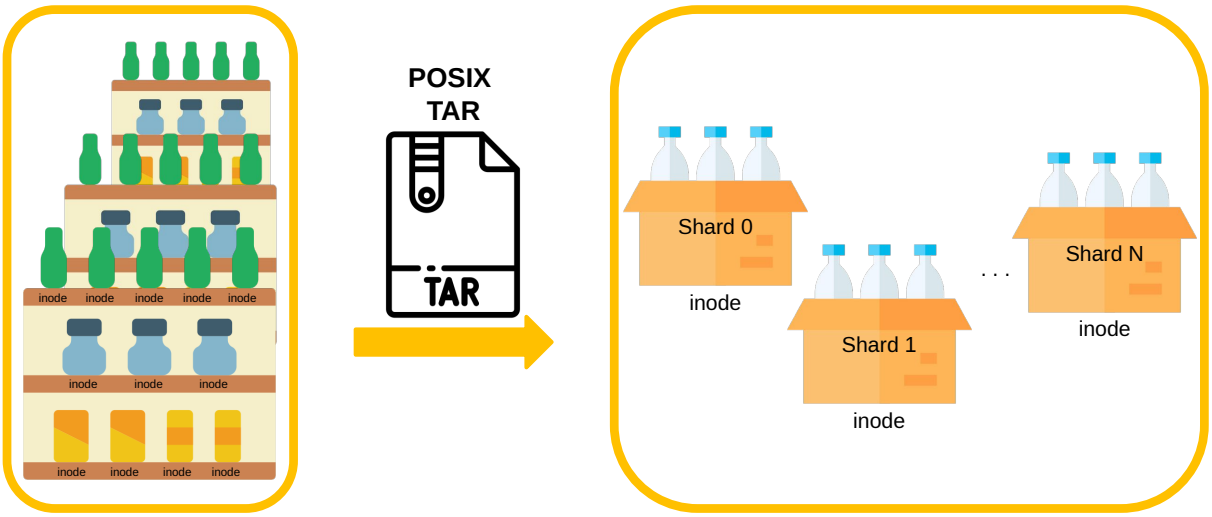
(TFRecord, Apache Parquet, HDF5, Zarr,...)

On Jean Zay, IDRIS recommends not storing more than 100k inodes per directory so as not to overload the parallel file system.

HuggingFace Hub also provides recommendations on dataset storage. The platform's success has led to a standardization of dataset formats that are worth following.

For large datasets, we recommend gathering several samples into a single file (archive). Standard dataset formats already exist, such as WebDataset.

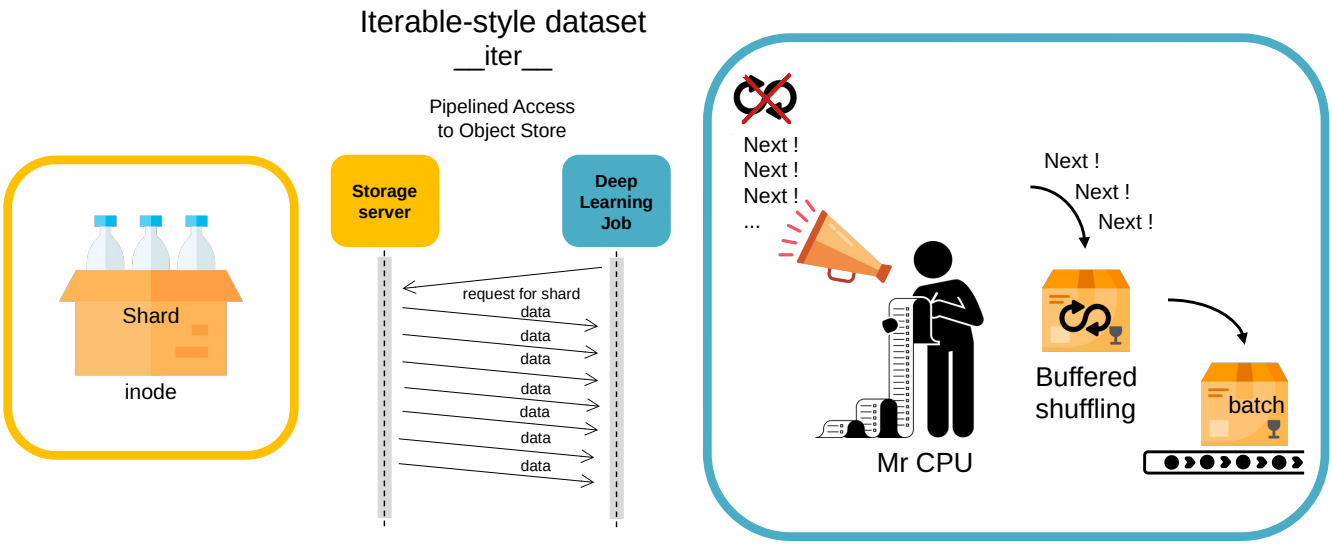
WebDataset format – Gathering data



Icon by Magnific

We present the WebDataset format in order to minimize the number of inodes.
The idea is to group the data in an archive format.

WebDataset format – Iterable dataset



Pros: Fewer I/Os, fewer inodes

Cons: Preprocessing, difficult to shuffle or distribute, unknown dataset length

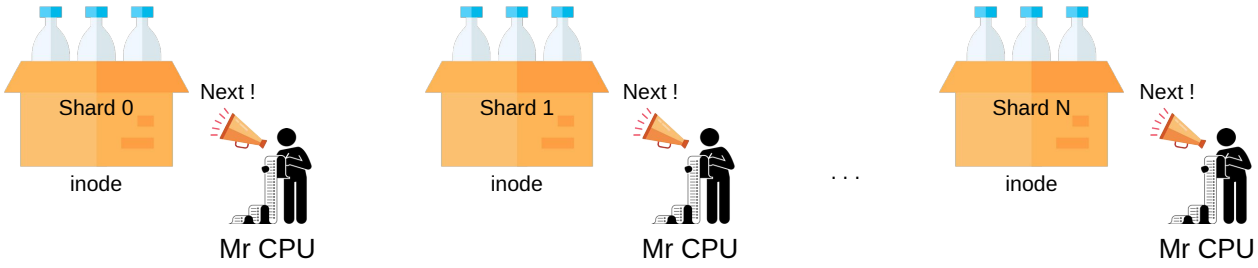
WebDataset is an iterable-style dataset. We access data contiguously. There is no mapping (idx, image).

This style of dataset is suited to databases whose structure is not known beforehand. This is the case here because the data is « hidden » in the archives.

Shuffling is managed by the CPU at buffer level. The size of this buffer is chosen by the user (proportionate to the batch size, for example).

WebDataset format - Sharding

Sharding is necessary to benefit from parallel implementation
(DataLoader multi-processing and Distributed Data Parallelism).



The number of shards should be a multiple of the number of tasks/GPUs.

1 shard = 1 archive.

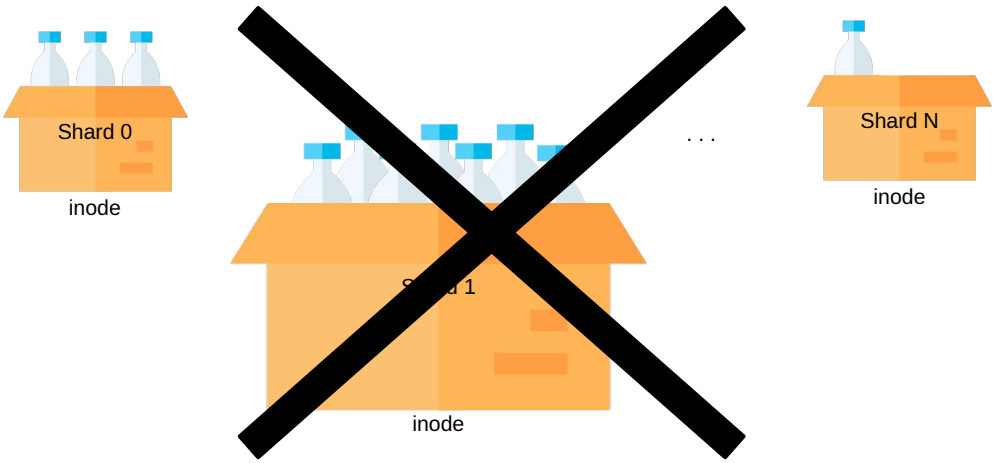
Sharding involves dividing the original dataset into multiple archives. A minimum number of archives is necessary to generate parallelism at the I/O level.

For example, 2 DataLoader workers will access two different archives.

Likewise, 2 GPUs must be supplied from different archives.

For example: nshards = ngpus x nworkers

WebDataset format - Sharding



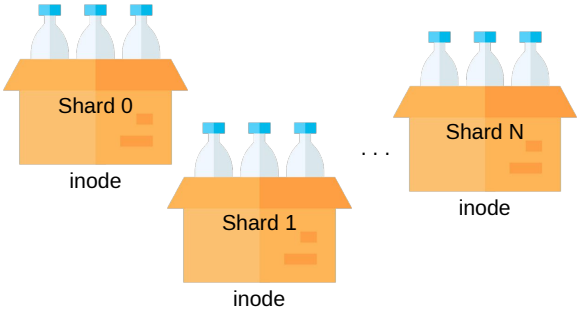
Samples must be evenly distributed among the shards to balance the workload between processes.

For the workload to be optimally distributed across the different processes, the data must be distributed as equally as possible in the archives.

WebDataset format - More or fewer shards?



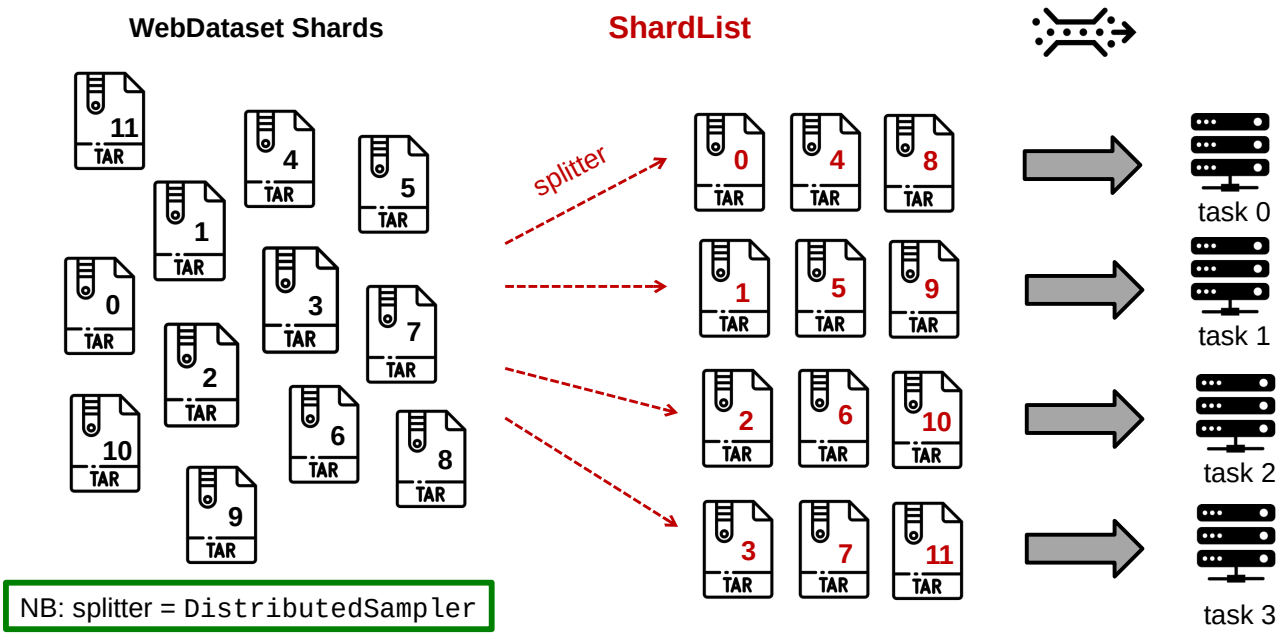
	More shards	Fewer shards
Large-scale distribution	👍	👎
CPU RAM usage	👍	👎
Inode quota	👎	👍
Number of I/O requests	👎	👍



The advantages and disadvantages of having a dataset with more or fewer shards.

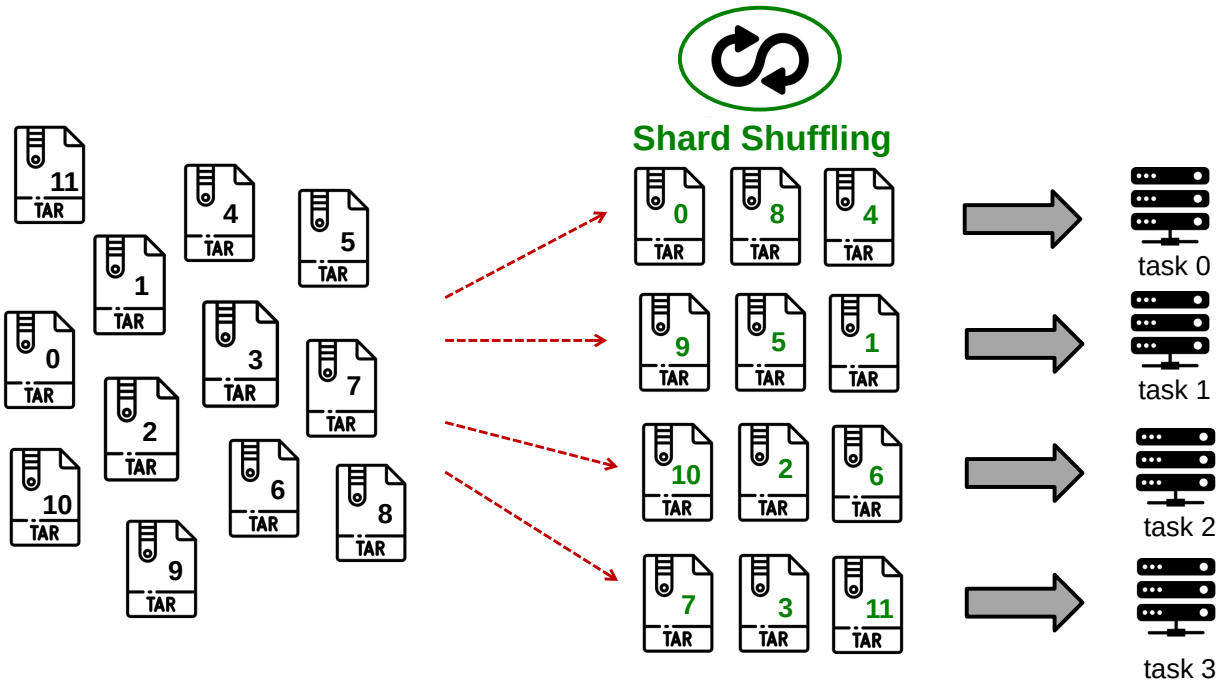
- The more shards you have, the more workers and GPUs you can run in parallel
- The more shards you have, the less memory each shard uses
- The fewer shards you have, the fewer inodes you store
- The fewer shards you have, the fewer I/O requests you perform

WebDataset – Multiworker sharding



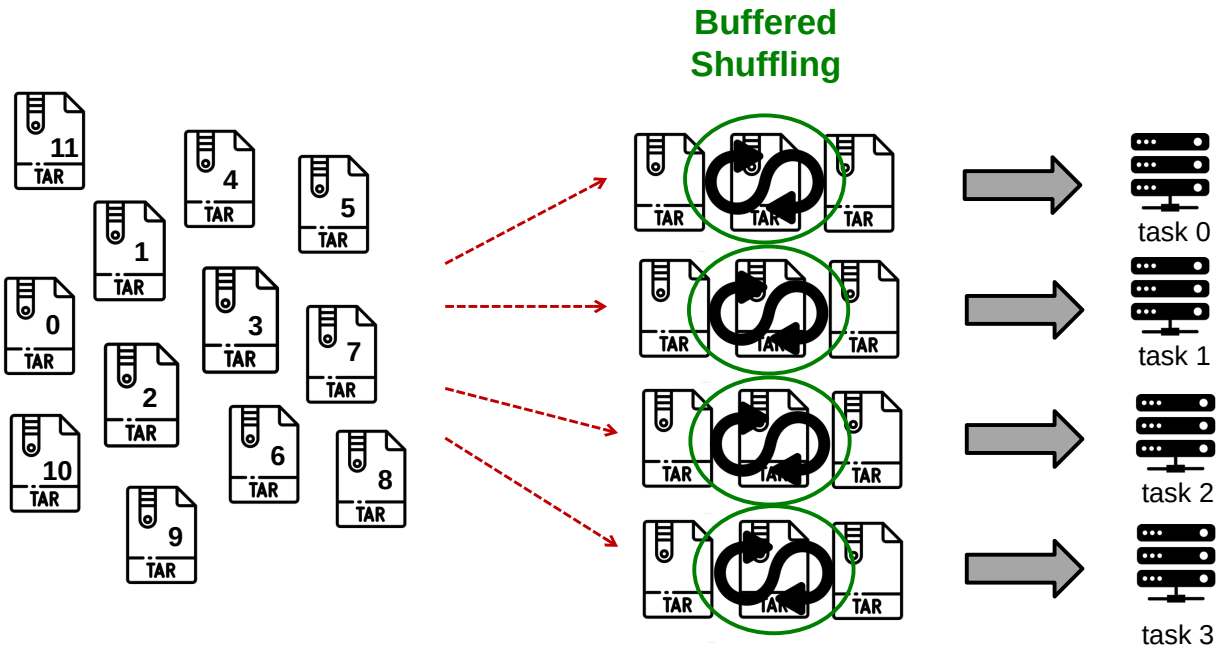
The shards are distributed on different processes according to the splitter function. This function replaces the DistributedSampler.

WebDataset - Shuffling



We can activate the first shuffling on the shard indexes. This shuffling is performed per process.

WebDataset - Shuffling

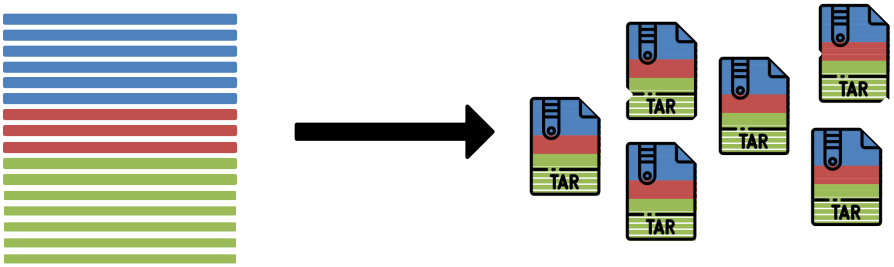


We can also activate a shuffling at the level of all data contained in a process.

Combining the two types of shuffling (shard shuffling and buffered shuffling) is the most effective method.

WebDataset - Preprocessing

- Distribute the samples as **evenly** as possible among the shards.
- Choose the number of shards **based on the number of GPUs and DataLoader workers** you will use.
- Distribute the samples so that each shard contains a **representative part of the dataset.**



+ Converting data before creating the archives to improve decoding performance?



Some advice for generating a dataset of the WebDataset type.

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```


Here is an example for using a WebDataset in your script. Details are provided below step by step.

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                               .shuffle(1000)\
                               .decode("pil")\
                               .to_tuple('jpg', 'cls')\
                               .map_tuple(transform, lambda x: x)\
                               .batched(mini_batch_size)\
                               .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

distribute shards among processes
(~DistributedSampler)

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                               .shuffle(1000)\
                               .decode("pil")\
                               .to_tuple('jpg', 'cls')\
                               .map_tuple(transform, lambda x: x)\
                               .batched(mini_batch_size)\
                               .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```



shuffling shards indexes per process

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

shuffling samples per process

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                               .shuffle(1000)\
                               .decode("pil")\
                               .to_tuple('jpg', 'cls')\
                               .map_tuple(transform, lambda x: x)\
                               .batched(mini_batch_size)\
                               .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

} description of shard content

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                               .shuffle(1000)\
                               .decode("pil")\
                               .to_tuple('jpg', 'cls')\
                               .map_tuple(transform, lambda x: x)\
                               .batched(mini_batch_size)\
                               .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

} transforming and batching

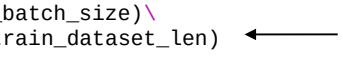
WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```



WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

batching handled by WebDataset class

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

usual DataLoader args

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```


drop_last equivalent

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```


define len(train_loader)

WebDataset - Performance test

I/O loop test: timing I/O requests and decoding operations

```
for i, (images,labels) in enumerate(loader):
    assert images.size()[0] == batch_size
```

Test performed on 1 V100 GPU (32GB)

- Loop over 10 iterations
- Calculation-free iterations & Minimum data transformation



Mean iteration time

We consider a very simple “I/O loop” test to compare performances while using a WebDataset or not.

The goal is to record the mean time needed for building a batch, which involves performing I/O requests and decoding the images.

We don’t perform any calculation in the loop and avoid transforming the images as much as possible since they are recorded as well in this test.

WebDataset - Performance test

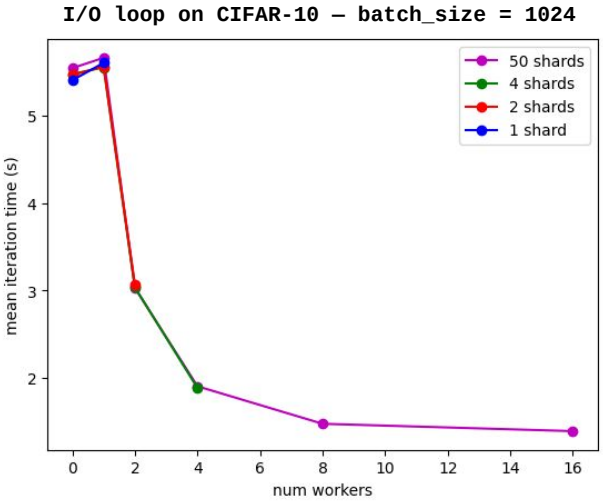
🤔 Can I work with only 1 shard?

Benchmark 1

Dataset: CIFAR-10 (50k images, no transformation)

Format: WebDataset, from 1 to 50 shards

Run: I/O loop while increasing the number of workers



- Sharding is necessary to benefit from parallel implementation (DataLoader multi-processing).

Test using the CIFAR-10 dataset.

Here we use a small dataset to rapidly generate WebDataset versions with more or fewer shards.

We observe that the more shards there are, the more we generate parallelism.

If your dataset is composed of fewer shards than workers, the code crashes with a clear error message.

WebDataset - Performance test

🤔 What do I get out of using WebDataset?

Benchmark 2

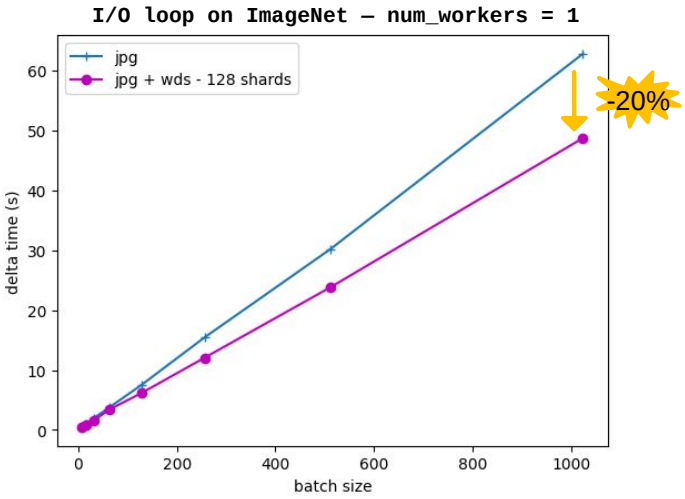
Dataset: ImageNet

- ~1.3M images
- *CenterCrop* is needed since images have variable sizes

Format:

- Original format: 1.3M JPG files (~140 GB)
- WebDataset with 128 shards (~213 GB)

Run: I/O loop while increasing the batch size



- 20% I/O speedup (with 1 worker)
- From 1.3M to 128 inodes for a similar storage volumetry

Test using the ImageNet dataset.

We consider the following two dataset formats:
- The original ImageNet format: 1.3M JPG files
- A WebDataset format with 128 shards (10k JPG files per shard)

We set *num_workers=1* and run the I/O loop while increasing the batch size.

Observations:
- The storage space needed to store the WebDataset is slightly larger due to additional metadata. It remains in the same order of magnitude.
- Using shards reduces the number of I/O requests, yielding a 20% speedup.
- The main benefit of using the WebDataset format is reducing the number of inodes.

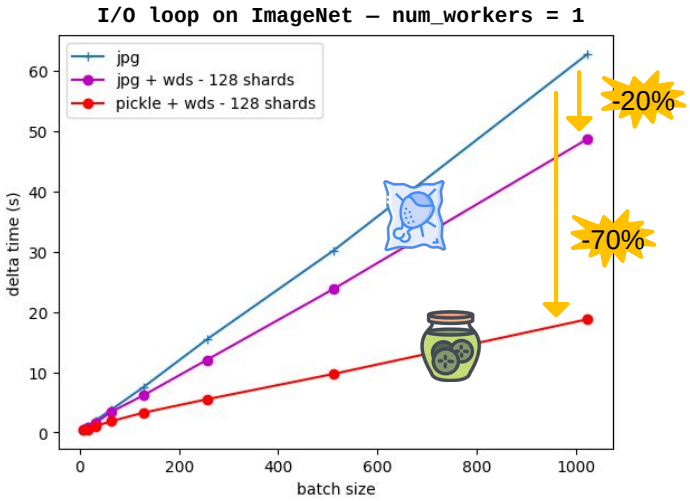
WebDataset - Performance test

🤔 Creating a WebDataset is time-consuming, why not store the data in binary format as well?

Benchmark 2 (extended)

Format:

- Original format: 1.3M JPG files (~140 GB)
- WebDataset with 128 shards (~213 GB)
- The same but with decoded samples (~826 GB)



- 70% I/O speedup (with 1 worker)
- From 1.3M to 128 inodes, but from 140 to 826 GB!

Test using the ImageNet dataset.

We consider an additional WebDataset with 128 shards, each containing 10k decoded images (images stored in Python pickle format).

We observe a 70% speedup at the cost of a significant increase in storage requirements (from 140 to 826 GB).

This illustrates the cost of dealing with compressed data formats mentioned earlier in this chapter.

WebDataset - Performance test

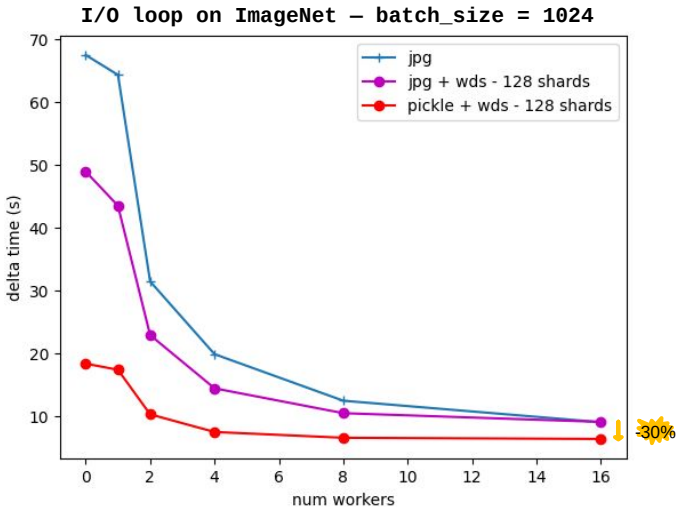
🤔 Does it scale up nicely?

Benchmark 3

Dataset: Same as above

Format: Same as above

Run: I/O loop while increasing the number of workers



- The WebDataset scales well
- The I/O speedup from sharding diminishes as the number of workers increases

Test using the ImageNet dataset

We set batch_size=1024 and run the I/O loop while increasing the number of workers.

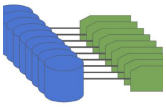
We observe that the WebDataset scales well.

In the case of ImageNet, the speedup from sharding (thus the reduction in I/O requests) disappears as the number of workers increases.

A 30% speedup remains when using the WebDataset with decoded samples.

Conclusion on dataset format

- For very large datasets, gathering several samples in shards is **mandatory** to minimize the number of inodes.
- Fewer inodes yield fewer I/Os. This may improve the loading performance but DataLoader standard optimizations (e.g. `num_workers`) are more efficient for this purpose.
- Standard formats already exist like WebDataset, TFRecord, Apache Parquet, HDF5, Zarr,...



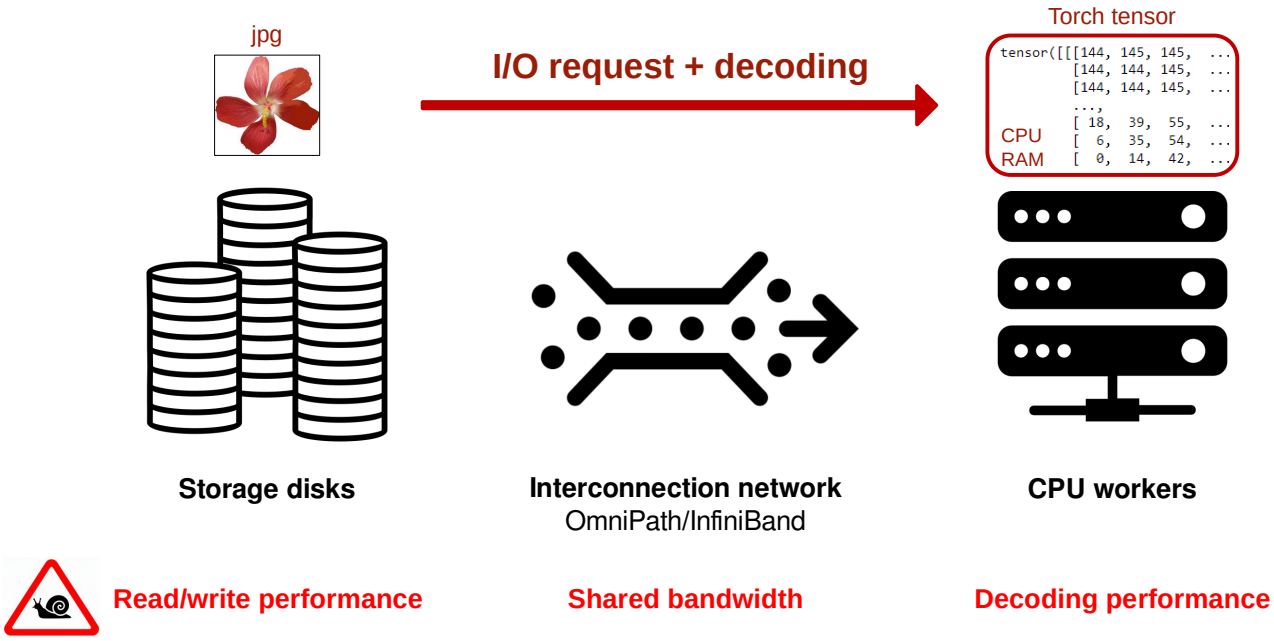








General conclusion - Context



Reminder: We aim to avoid three main obstacles: Slow I/O read speeds on disks, bandwidth sharing in the interconnection network, and the cost of input data decoding on the CPU.

General conclusion – Takeaways



Where should I store my dataset?



No automatic deletion

Best I/O performance
Very large quota

Which format for my data?



Fast decoding

Low storage usage

Which format for my dataset?



Map-style dataset
Easy to create and work with

Significantly fewer inodes
Fewer I/Os → potential speedup

This slide summarizes the pros and cons of each option, considering the three key questions:

– Where should I store my dataset?
– Which format for my data?
– Which format for my dataset?